CS 475 – Distributed Systems

Project 3: RESTful API

Team: **[Awn Abdullah Ahmad Ersheidat 165827]**
[Mohammad-noor Ahmad Helael Al-khaldi 164925]
[Mohammad Saleh Bassam Hijazi 163246]

Task 1 – Main Resources

Resource	Description	Base URL
Users	Registered accounts	/api/users
Auth	Login and tokens	/api/auth
Books	Books in the store	/api/books
Cart	The user's shopping cart	/api/cart
Orders	Placed orders	/api/orders

Task 2 & 3 – Endpoints and HTTP Methods

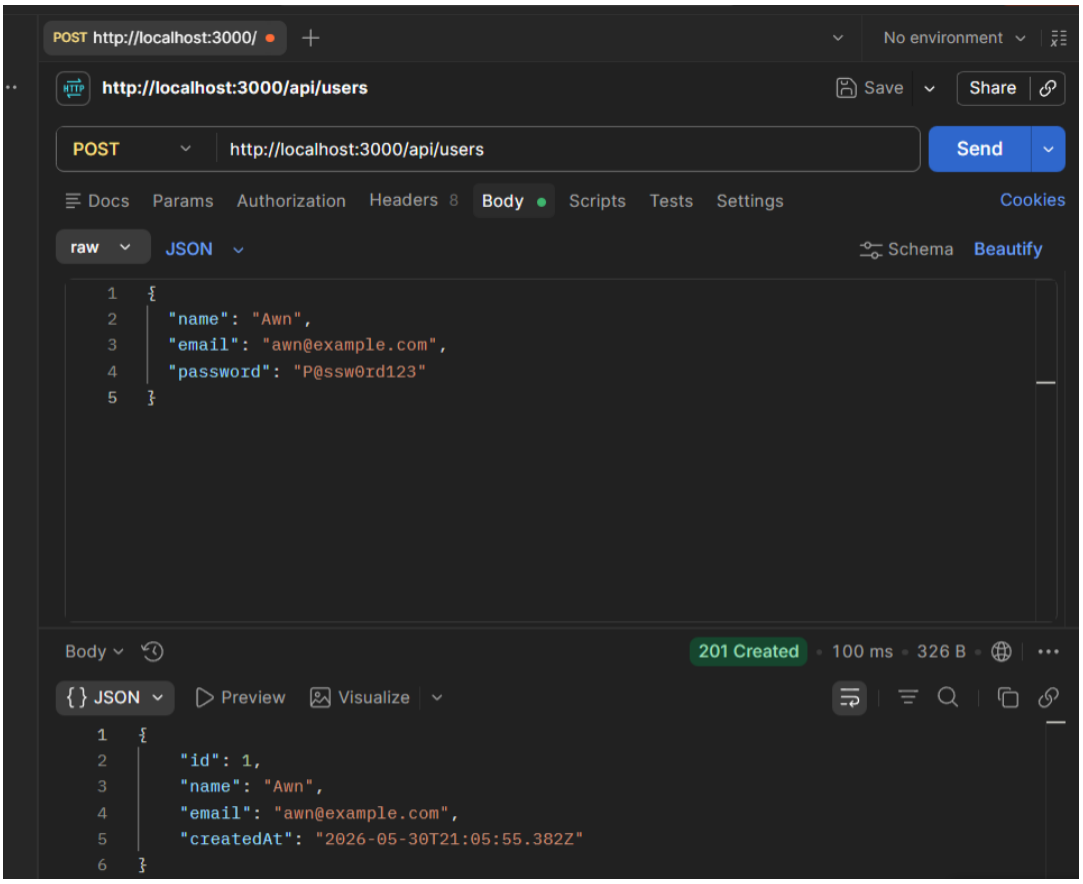
Method	Endpoint	Purpose
POST	/api/users	Create a new account
POST	/api/auth/login	Log in and get a token
GET	/api/books	View all books
GET	/api/books?title=&author=&category=	Search books
GET	/api/books/{id}	View one book
POST	/api/cart/items	Add a book to the cart
DELETE	/api/cart/items/{bookId}	Remove a book from the cart
GET	/api/cart	View the cart
POST	/api/orders	Place an order
GET	/api/orders	View order history
GET	/api/orders/{id}	View one order
DELETE	/api/orders/{id}	Cancel an order

Task 4 – Sample Requests and Responses

Five endpoints tested in Postman. Each screenshot also shows the status code (Task 5).

1) POST /api/users – Create user

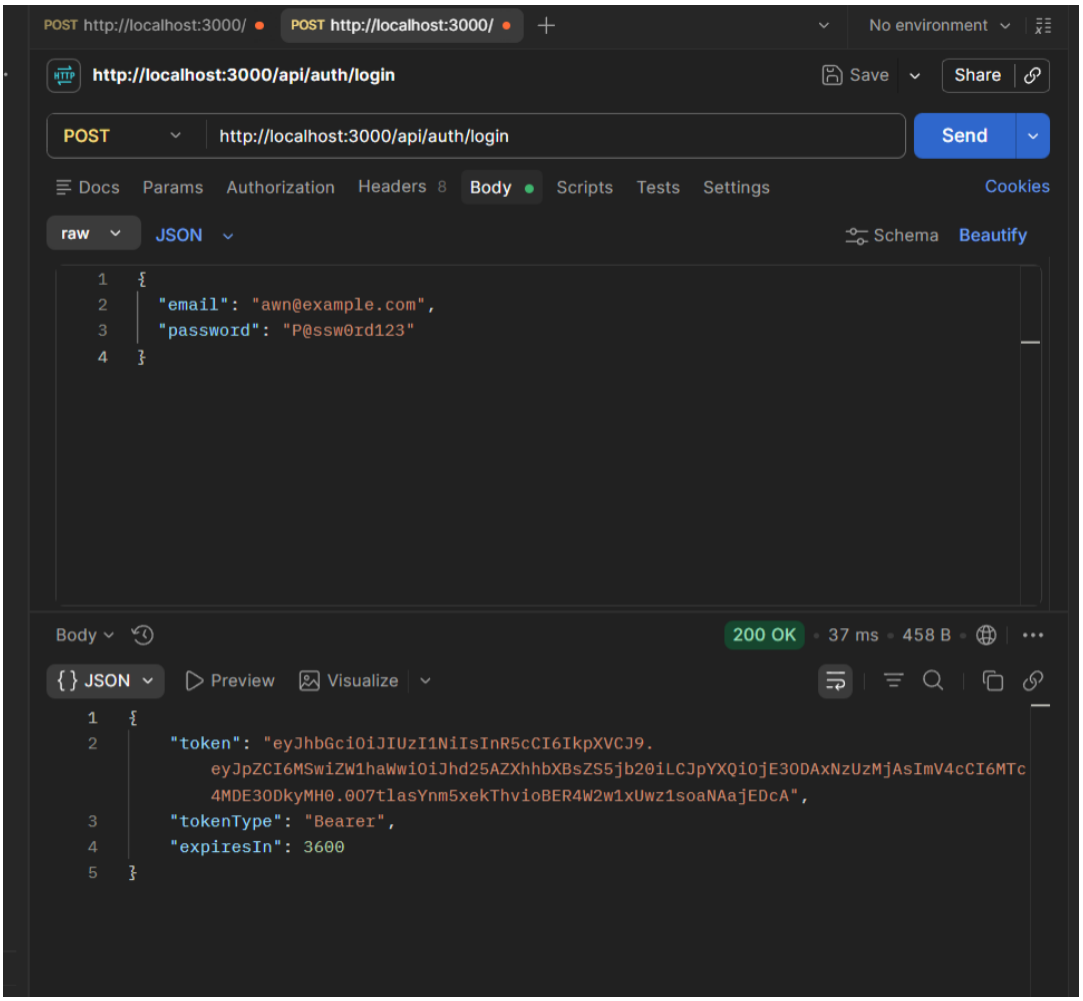
```
Request:
{
  "name": "Awn",
  "email": "awn@example.com",
  "password": "P@ssw0rd123"
}
```



201 Created – user created (password not returned).

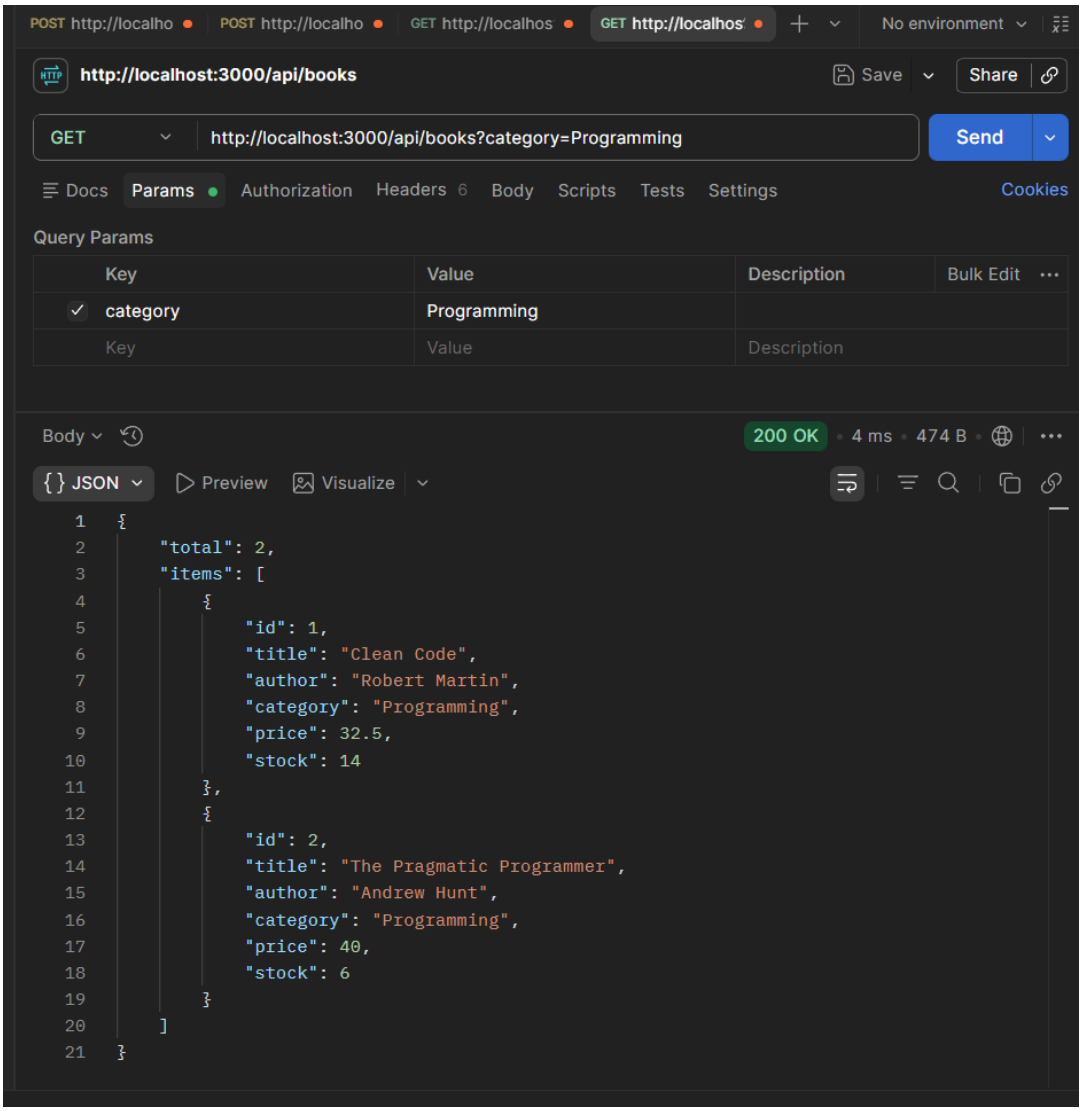
2) POST /api/auth/login – Log in

```
Request:
{
  "email": "awn@example.com",
  "password": "P@ssw0rd123"
}
```



200 OK – token returned.

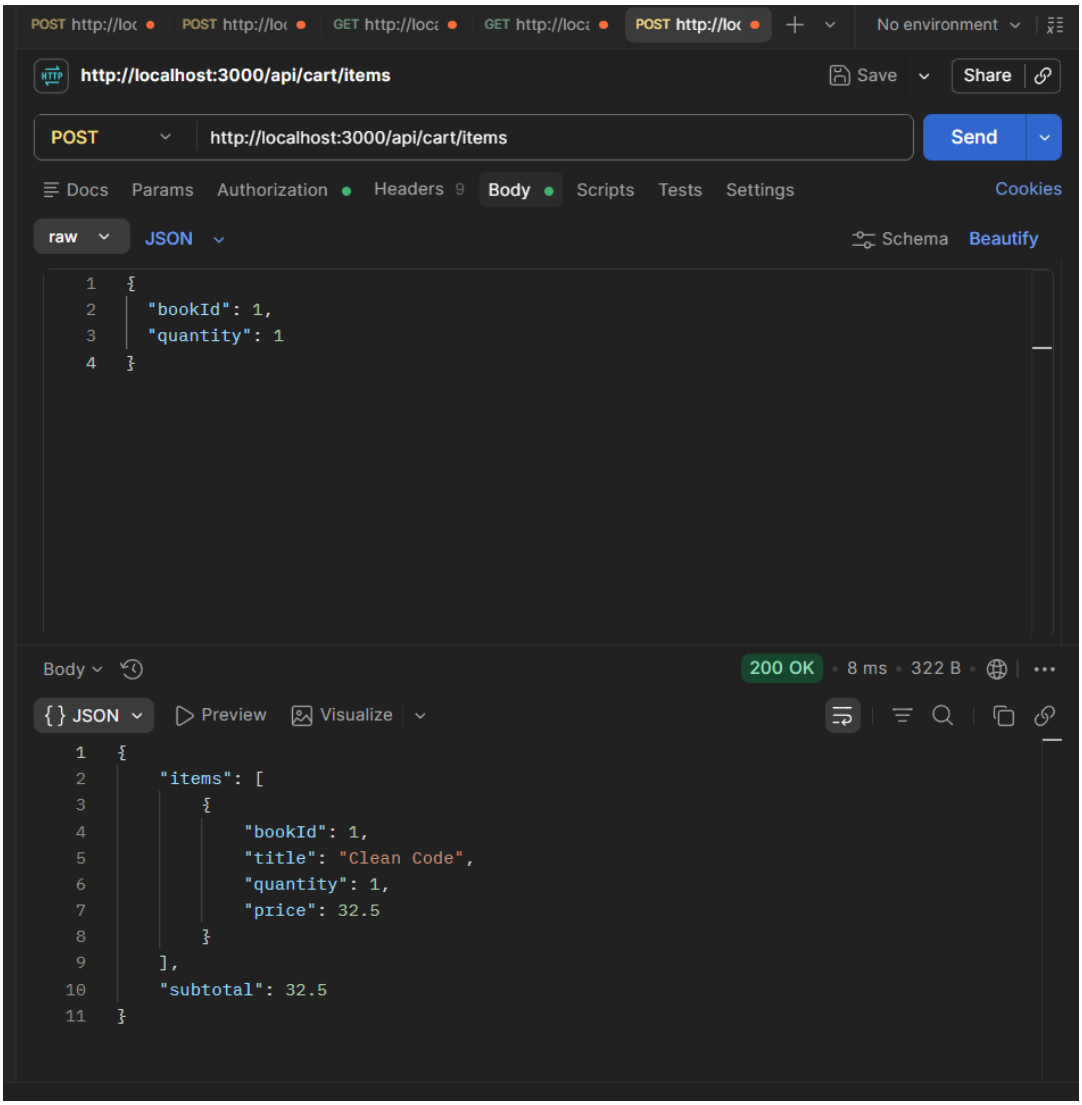
3) GET /api/books?category=Programming – Search



200 OK – books filtered by category.

4) POST /api/cart/items – Add to cart (needs token)

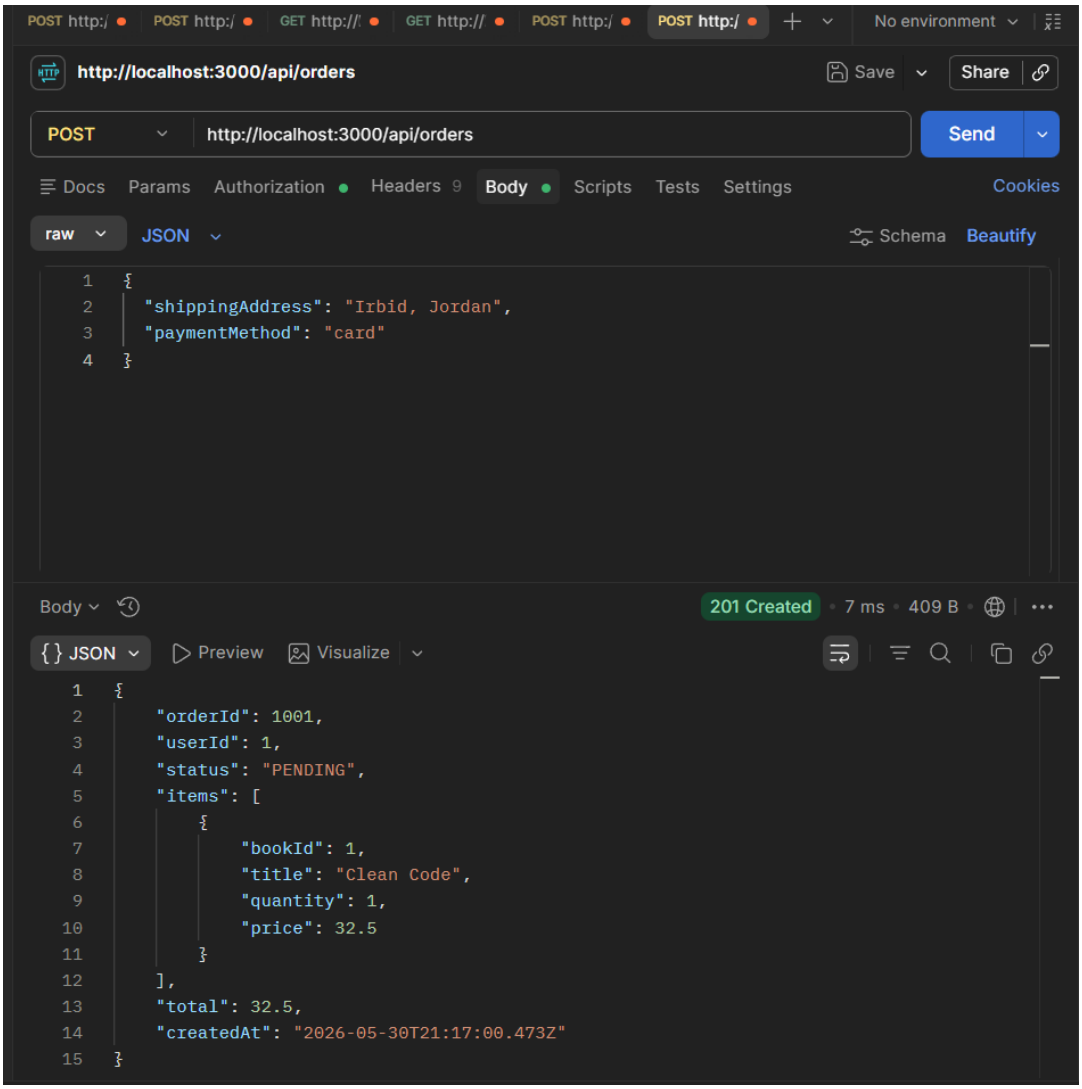
```
Request:
{
  "bookId": 1,
  "quantity": 1
}
```



200 OK – book added to the cart.

5) POST /api/orders – Place order (needs token)

```
Request:
{
  "shippingAddress": "Irbid, Jordan",
  "paymentMethod": "card"
}
```

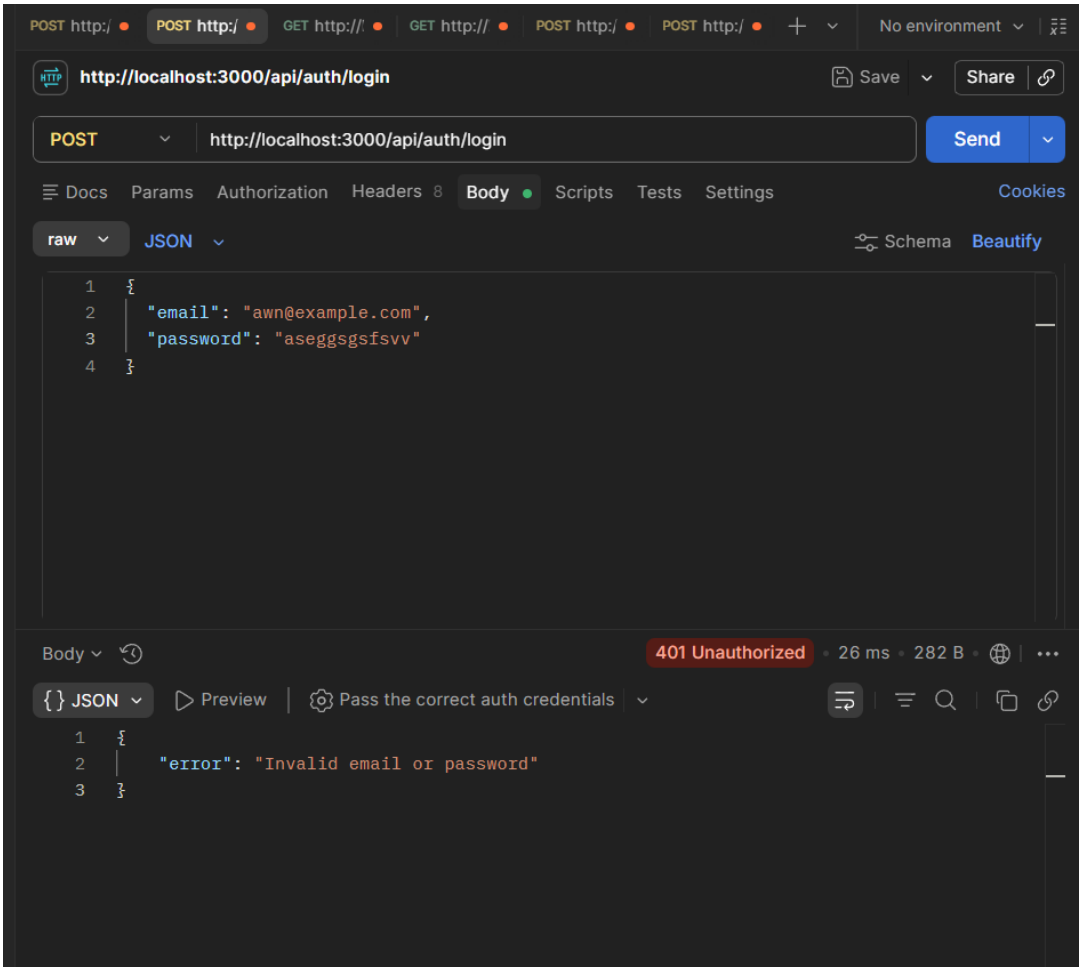


201 Created – order placed with status PENDING.

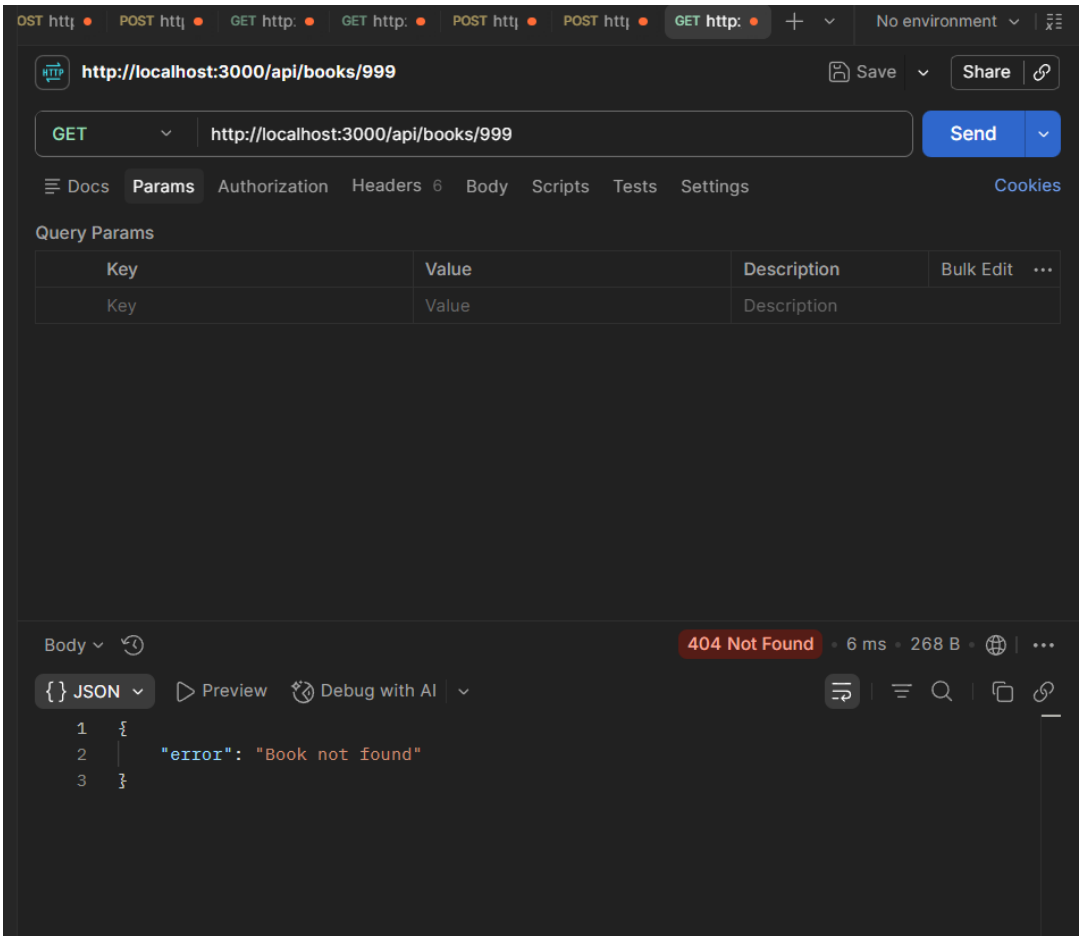
Task 5 – HTTP Status Codes

Code	Meaning	When used
200 OK	Success with data	Login, view/search books, view/add cart
201 Created	Resource created	Register user, place order
204 No Content	Success, no body	Remove from cart, cancel order
400 Bad Request	Invalid input	Missing fields in body
401 Unauthorized	Not authenticated	No/invalid token, wrong password
403 Forbidden	Not allowed	Opening another user's order
404 Not Found	Does not exist	Book or order id not found
409 Conflict	Conflict	Email already registered
422 Unprocessable	Wrong values	Quantity < 1, empty cart on order
500 Server Error	Server failure	Unexpected error

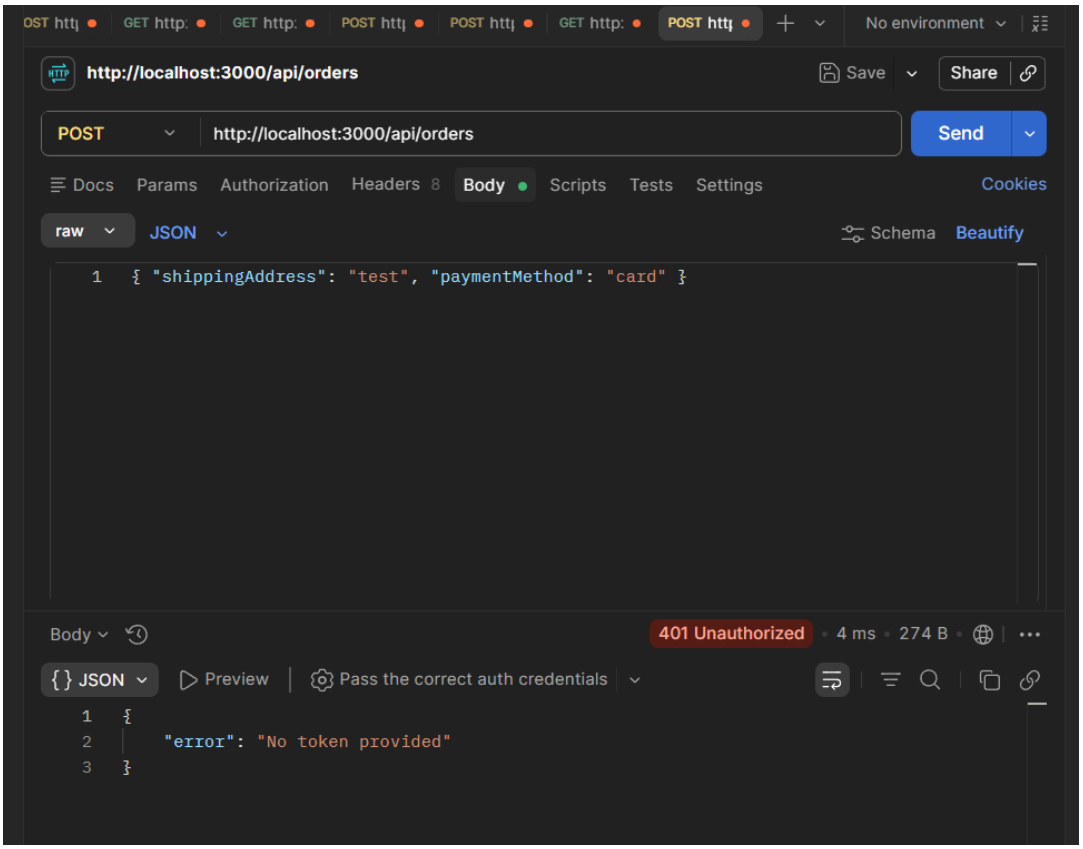
Failure cases tested in Postman:



401 Unauthorized – wrong password.



404 Not Found – book id does not exist.



401 Unauthorized – protected endpoint with no token.

Task 6 – Authentication

Authentication uses JWT (JSON Web Token), which is stateless and fits a distributed system. The steps:

1. On register, the password is hashed with bcrypt before being stored (never plain text).
2. On login, the server checks the password against the hash and returns a signed token containing the user id and an expiry (1 hour).
3. For protected requests, the client sends the token in the header: Authorization: Bearer <token>.
4. The server verifies the token's signature and expiry on each request. Valid → request continues; invalid → 401 Unauthorized.

Task 7 – Security Issues and Prevention

1) SQL / Injection

- Risk: malicious input in fields manipulates database queries.
- Prevention: use parameterized queries / an ORM, and validate all input.

2) Broken authentication / stolen token

- Risk: weak passwords or leaked tokens let attackers impersonate users.
- Prevention: hash passwords with bcrypt, use HTTPS, short token expiry, and rate-limit the login endpoint.

3) Broken access control

- Risk: a user requests another user's order and sees their data.
- Prevention: check that the resource's owner matches the user id in the token; return 403 Forbidden if not.